

SIGNUM

Sistema de Reconocimiento de Lengua de Señas Mexicana

Exposición: Aplicaciones Web Orientadas a Servicios

El Problema

- **2.3 millones** de personas con discapacidad auditiva en México
- Solo **~5,000** intérpretes certificados de LSM
- Las apps existentes de traducción LSM requieren hardware especializado
- No existe una plataforma **colaborativa** donde la comunidad pueda entrenar y compartir modelos

SIGNUM resuelve esto con una arquitectura SOA desde cero

Visión General de SIGNUM

- Traducción de señas en **tiempo real** usando cámara del dispositivo
- **Inferencia 100% local**: MediaPipe + Random Forest en JavaScript puro
- **Colaborativo**: usuarios pueden subir muestras y entrenar modelos compartidos
- **Multidispositivo**: app web + app móvil nativa (Expo)
- **3 tipos de señas**: letras, palabras estáticas y señas dinámicas

La predicción funciona **sin conexión a internet**

Arquitectura Orientada a Servicios

Error 404: Not Found

Servicio 1: Frontend Web (Next.js)

Aspecto	Detalle
Framework	Next.js 16 + React 19
Hosting	Vercel (serverless)
Cámara	<code>getUserMedia</code> + MediaPipe Tasks-Vision CDN
Inferencia	RandomForest en JS puro (cliente)
TTS	Web Speech API + ElevenLabs (proxy servidor)
Almacenamiento	IndexedDB local
API Routes	POST <code>/api/ai/complete</code> , GET <code>/api/tts/elevenlabs</code>

- La app web es un **servicio independiente** que corre 100% en el navegador

Servicio 2: App Móvil (Expo/React Native)

Aspecto	Detalle
Framework	Expo SDK 54 + React Native 0.81
Plataformas	iOS + Android
Cámara	WebView con HTML de MediaPipe embebido
Inferencia	RandomForest JS puro
TTS	expo-speech nativo
Almacenamiento	expo-sqlite local (WAL mode)
Auth	Supabase JWT con AsyncStorage

- Misma lógica de predicción que el frontend, implementación nativa distinta

Servicio 3: ML Service (Express)

Aspecto	Detalle
Runtime	Node.js + Express 4
Puerto	8000
Hosting	Render
Endpoints	GET /ml/health , POST /ml/train

POST /ml/train — Entrena un Random Forest colaborativo:

1. Lee muestras de Supabase (`muestras_entrenamiento`)
2. Split 80/20 train/test
3. Entrena RF (50 árboles, profundidad 15)
4. Evalúa: accuracy, precision, recall, F1, matriz de confusión

Servicio 4: Supabase (Backend-as-a-Service)

Capa	Descripción
Auth	Email/password con JWT
Base de datos	PostgreSQL con esquema relacional
Storage	S3-compatible (bucket modelos)
RLS	Row Level Security en todas las tablas
Triggers	Auto-crear perfil al registrarse

Tablas principales: usuarios , traducciones , avances , muestras_entrenamiento ,
modelos

Supabase actúa como **servicio de datos central** compartido por todos los clientes

Servicios Externos

Groq (LLM)

```
POST https://api.groq.com/openai/v1/chat/completions  
Modelo: Llama 4 Scout 17B  
Propósito: completar frases (gloss → español natural)  
Proxy: API key oculta en servidor Next.js
```

ElevenLabs (TTS)

```
GET https://api.elevenlabs.io/v1/text-to-speech/{voiceId}  
Modelo: eleven_multilingual_v2  
Propósito: voz natural para frases completadas  
Proxy: API key oculta en servidor Next.js
```

CRITERIO 1

Autonomía del Servicio

Peso: 25%

Autonomía del Servicio

Cada servicio puede ejecutarse y desplegarse de forma independiente, sin depender del código interno o del proceso del otro.

Servicio	¿Funciona solo?	Evidencia
Frontend Web	✓	Inferencia local, sin backend obligatorio
App Móvil	✓	Misma inferencia local, SQLite local
ML Service	✓	Lee de Supabase, no del frontend
Supabase	✓	Gestionado externamente, no acoplado al código

Ningún servicio comparte código entre sí: cada uno tiene su propio repositorio lógico
(`frontend/`, `mobile/`, `ml-service/`)

Autonomía del Servicio — Evidencia

Prueba de independencia:

1. **Frontend sin ML Service:** la predicción es local, no requiere el server. Solo no hay modelos colaborativos.
2. **Frontend sin Supabase:** la cámara y predicción siguen funcionando. Solo no se persisten traducciones ni hay login.
3. **Mobile sin backend:** igual que el frontend, todo es local.
4. **ML Service sin frontend:** se puede invocar con `curl POST /ml/train` → entrena, evalúa y sube a Supabase. No necesita que ningún cliente esté activo.

```
# ML Service funciona de forma totalmente autónoma
curl -X POST http://localhost:8000/ml/train \
  -H "Content-Type: application/json" \
  -d '{"type": "letter", "activate": true}'
```

CRITERIO 2

Contrato de Servicio Bien Definido

Peso: 20%

Contrato: Frontend API Routes

POST /api/ai/complete

```
Request: { "phrase": "yo comer manzana" }  
Response: { "completed": "Yo como una manzana." }
```

- Entrada: frase en notación "gloss" (secuencia de palabras sin gramática)
- Salida: frase corregida en español mexicano natural
- Modelo: `meta-llama/llama-4-scout-17b-16e-instruct` , temp 0.2, max_tokens 200

GET /api/tts/elevenlabs?text={}&voice_id={}

```
Parámetros: text (string, máx 500 chars), voice_id (whitelist)  
Response:    audio/mpeg binario
```

Contrato: ML Service API

POST /ml/train

// Request

```
{  
  "type": "letter" | "word" | "dynamic",  
  "hyperparams": { "nTrees": 50, "maxDepth": 15, "minSamplesLeaf": 2 },  
  "activate": false  
}
```

// Response

```
{  
  "success": true,  
  "version": "letter_v3",  
  "type": "letter",  
  "metrics": {  
    "accuracy": 0.85, "precisionAvg": 0.83,  
    "recallAvg": 0.81, "f1Score": 0.82  
  },  
  "..."  
}
```

Contrato: Supabase

Tabla `muestras_entrenamiento` (datos de entrenamiento)

```
id_muestra SERIAL PRIMARY KEY,  
id_usuario UUID REFERENCES auth.users,  
tipo TEXT CHECK (tipo IN ('letter', 'word', 'dynamic')),  
etiqueta TEXT NOT NULL,  
landmarks JSONB NOT NULL,    -- [63 floats] normalizados  
created_at TIMESTAMPTZ DEFAULT now()
```

Tabla `modelos` (metadatos de modelos)

```
id_modelo SERIAL PRIMARY KEY,  
tipo TEXT NOT NULL,  
version TEXT NOT NULL UNIQUE,    -- "letter_v3"  
storage_path TEXT NOT NULL,      -- ruta en Supabase Storage  
accuracy NUMERIC(5,4),  
n_trees INTEGER, n_features INTEGER,  
clases TEXT[],
```

CRITERIO 3

Bajo Acoplamiento

Peso: 10%

Bajo Acoplamiento

La comunicación entre servicios debe darse únicamente a través de su interfaz pública.

Par de servicios	Acoplamiento	Canal
Frontend ↔ Supabase	Solo SDK público	HTTPS + JWT
Mobile ↔ Supabase	Solo SDK público	HTTPS + JWT
ML Service ↔ Supabase	Solo SDK público	HTTPS + SRK
Frontend ↔ Groq	Solo body JSON	Proxy Next.js
Frontend ↔ ML Service	No hay comunicación directa	—

- **Los clientes nunca consultan al ML Service** para predecir; la inferencia es local
- **El ML Service nunca consulta a los clientes**; solo lee/escribe Supabase

Bajo Acoplamiento — Evidencia en Código

Frontend no importa nada del ML Service:

```
// frontend/src/services/rf-inference.ts
// RandomForestPredictor corre ENTERAMENTE en el navegador
// No hace fetch al ML Service
// La URL "ws://localhost:8000/ws/predict" es un fallback inactivo
// que ni siquiera tiene endpoint implementado en ml-service
```

ML Service no importa nada del frontend:

```
// ml-service/src/server.js
// Lee de supabase directamente con SERVICE_ROLE_KEY
const supabase = createClient(url, serviceRoleKey, {
  auth: { autoRefreshToken: false, persistSession: false }
});
// No tiene ruta que reciba landmarks del frontend
```

CRITERIO 4

Mecanismo de Seguridad entre Servicios

Peso: 10%

Seguridad — Autenticación

Supabase Auth (JWT)

- Registro/login con email y contraseña
- JWT gestionado automáticamente por `@supabase/supabase-js`
- **Frontend:** sesión en localStorage del navegador
- **Mobile:** sesión en AsyncStorage (RN)

```
// Login: JWT generado y adjuntado a todas las requests
const { data, error } = await supabase.auth.signInWithPassword({
  email, password
});
// El SDK automáticamente incluye Authorization: Bearer <jwt>
```

ML Service — Autenticación server-to-server

```
// Usa SERVICE_ROLE_KEY para bypass RLS
```

Seguridad — Autorización (RLS)

Todas las tablas de Supabase tienen **Row Level Security**:

Tabla	Política
usuarios	Solo ver/editar tu propio perfil
traducciones	Solo insertar/ver las tuyas
muestras_entrenamiento	Insertar tuyas, ver todas
modelos	Ver todos (auth), modificar solo admin/service_role
evaluaciones	Insertar tuya, admin ve todas

```
-- Ejemplo de política RLS en muestras_entrenamiento
CREATE POLICY "Usuarios insertan sus muestras"
ON muestras_entrenamiento FOR INSERT
WITH CHECK (auth.uid() = id_usuario);
```

Seguridad — Protección de API Keys

Clave	Dónde vive	¿Expuesta?
GROQ_API_KEY	<code>.env.local</code> → servidor Next.js	✗ No — proxy server-side
ELEVENLABS_API_KEY	<code>.env.local</code> → servidor Next.js	✗ No — proxy server-side
SUPABASE_SERVICE_ROLE_KEY	<code>ml-service/.env</code> → Render	✗ No — solo server
EXPO_PUBLIC_GROQ_API_KEY	<code>.env</code> → bundle app	⚠ Sí (trade-off aceptado)

Estrategia de proxy en Next.js:

```
// app/api/ai/complete/route.ts
export async function POST(req: Request) {
  const { phrases } = await req.json();
```

Seguridad — Manejo de fallos de autenticación

Frontend (auth.service.ts)

```
export async function login(email: string, password: string) {  
  const { data, error } = await supabase.auth.signInWithPassword({ email, password });  
  if (error) {  
    if (error.message.includes("Invalid login")) {  
      throw new Error("Credenciales inválidas");  
    }  
    if (error.status === 429) {  
      throw new Error("Demasiados intentos. Espera unos minutos.");  
    }  
    throw error;  
  }  
  return data.user;  
}
```

- Errores específicos para cada caso de fallo
- Rate limiting de Supabase (HTTP 429) manejado explícitamente

CRITERIO 5

Recursos y Calidad de la Exposición

Peso: 10%

Diagrama de Arquitectura

Dirección real de la comunicación entre servicios

A large, blurred, red text graphic that reads "Error 404: Not Found". The text is set against a light pink background and is partially obscured by a thick red vertical bar on the left side of the slide.

Demostración en Vivo

Plan de demostración:

1. **Registro/Login** en Supabase Auth
2. **Cámara activa** — MediaPipe detecta la mano en tiempo real
3. **Predicción local** — Random Forest clasifica letra/palabra/seña
4. **Auto-añadir** — la letra se agrega a la frase
5. **Completar frase** — Groq convierte gloss a español natural
6. **TTS** — ElevenLabs reproduce la frase

Video alternativo preparado por si algo falla

CRITERIO 6

Defensa Técnica

Peso: 25%

¿Por qué inferencia local y no en servidor?

Enfoque	Latencia	Offline	Escalabilidad	Privacidad
Inferencia en servidor	200-500ms por frame	✗ No funciona sin red	Requiere servidor GPU	Landmarks salen del dispositivo
Inferencia local (SIGNUM)	<5ms por frame	✓ Funciona sin internet	Escala con cada dispositivo	Datos nunca salen

- $63 \text{ floats} \times 30 \text{ fps} \times N \text{ usuarios}$ = inviable en servidor
- MediaPipe corre en GPU del navegador vía WebGL/WASM
- Modelo RandomForest pesa pocos KB, se entrena en <1 segundo
- Solo se suben muestras a Supabase cuando el usuario decide colaborar

¿Por qué 4 servicios separados?

Decisión	Justificación
Frontend ≠ Mobile	Diferentes APIs de cámara, TTS, almacenamiento local. Comparten inferencia pero no UI.
ML Service separado	El entrenamiento colaborativo requiere leer toda la base de datos (costoso). No debe bloquear a los clientes.
Supabase como BaaS	PostgreSQL gestionado con Auth y Storage integrado. Evita montar y mantener un servidor de base de datos propio.
Groq/ElevenLabs externos	LLMs requieren GPU especializada. Más barato y rápido usar APIs existentes que hostear modelos.

Separamos lo que escala distinto, lo que falla distinto y lo que evoluciona distinto.

¿Qué pasa si un servicio deja de estar disponible?

Servicio caído	Impacto	¿El sistema sigue funcionando?
Supabase	Sin login, sin historial, sin modelos colaborativos	✅ Sí — cámara y predicción local intactas
ML Service	Sin entrenamiento colaborativo	✅ Sí — entrenamiento local sigue disponible
Groq API	Sin completar frases con IA	✅ Sí — el fraseador manual y TTS nativo siguen
ElevenLabs API	Sin voz premium	✅ Sí — Web Speech API / expo-speech como fallback
Vercel (hosting web)	App web inaccesible	⚠️ La app web no carga; la app móvil sigue al 100%

¿Por qué Random Forest y no Red Neuronal?

Random Forest (elegido)

Ventaja	Detalle
Velocidad de entrenamiento	<1 segundo en JS, viable en navegador y móvil
Tamaño del modelo	Pocos KB serializado como JSON
Inferencia	$O(N_{\text{trees}} \times \text{depth})$, ~50k operaciones por predicción
Sin dependencias	JS puro, no requiere TensorFlow.js, ONNX ni WASM extra
Interpretabilidad	Feature importance por Gini, comprensible

Seguridad: Decisiones de Diseño

Decisión	¿Por qué?
JWT + RLS	Cada usuario solo accede a sus datos. Las políticas viven en la DB, no en el código de la app.
Proxy server-side	Las API keys de Groq y ElevenLabs nunca se exponen al navegador.
SERVICE_ROLE_KEY	El ML Service necesita leer todas las muestras para entrenar. Bypassa RLS de forma controlada.
Whitelist de voces TTS	Evita que un atacante use el proxy para consumir voces premium fuera del tier gratuito.
Triggers SQL	<code>handle_new_user()</code> auto-crea el perfil al registrarse. La lógica vive en la DB, no en el código.
Guest User	

Autonomía: ¿está garantizada realmente?

Prueba de autonomía real:

1. Detener el ML Service (kill process)
 - Frontend y Mobile siguen prediciendo normalmente ✓
 - Solo se pierde el endpoint POST /ml/train ⚠
2. Detener Supabase (simular caída de red)
 - Cámara + predicción local funcionan ✓
 - Login no disponible, datos no se persisten ⚠
3. Detener Groq API
 - El botón "Completar frase" falla
 - Web Speech API sigue funcionando para TTS ✓
 - expo-speech nativo en mobile sigue funcionando ✓
4. Detener Vercel (frontend web caído)
 - La app móvil de Expo sigue 100% funcional ✓

No hay single point of failure para la funcionalidad principal

Contratos: Interfaz pública estricta

Todos los servicios se comunican exclusivamente mediante **interfaces públicas documentadas**:

Servicio	Interfaz pública	Sin acceso a
Frontend	API routes (HTTP POST/GET)	Código del ML Service
Mobile	Supabase SDK + API Groq directa	Código del frontend web
ML Service	Endpoints REST (HTTP POST/GET)	Variables de entorno de los clientes
Supabase	PostgreSQL + Storage + Auth (SDK)	Lógica de negocio de ningún cliente

```
// Ejemplo: el frontend NO sabe cómo entrena el ML Service
// Solo conoce el contrato del modelo RF serializado
const model = await db.getModel("rf-letter");
rfLetter.current.loadFromModel(model.data);
```

Bajo Acoplamiento — ¿Puede un servicio cambiar sin romper otros?

Escenarios de cambio independiente:

Cambio	Servicios afectados	¿Rompe a otros?
Cambiar UI del frontend (Tailwind → CSS Modules)	Solo frontend web	✗ No
Cambiar TTS de expo-speech a otra librería	Solo mobile	✗ No
Cambiar RF hyperparámetros en ML Service	Solo ML Service	✗ No (el contrato es el modelo JSON)
Agregar tabla a Supabase	Solo Supabase	✗ No (clientes consultan por

Conclusión

SIGNUM cumple con los 6 criterios SOA:

Criterio	Peso	Cumplimiento
Autonomía del servicio	25%	✓ 4 servicios independientes, 0 código compartido
Contrato bien definido	20%	✓ API REST + esquema SQL públicos
Bajo acoplamiento	10%	✓ Solo comunicación vía interfaz pública
Seguridad entre servicios	10%	✓ JWT + RLS + API key proxy + CSP
Recursos de exposición	10%	✓ Diagrama + demo en vivo + video backup
Defensa técnica	25%	✓ Decisiones justificadas, tolerancia a fallos

¿Preguntas?

Gracias

"Conecta con el mundo usando Lengua de Señas Mexicana."